

MINI ASSESSMENT

API Error Handling and Safe JSON Access

Name	E. Rama Tulasi
Register No.	99230040298
Department	Computer Science and Engineering

1. Aim

To study the principles of robust error handling while consuming web APIs, and to apply safe techniques for accessing values inside JSON responses so that an application does not crash when the data returned is missing, malformed or of an unexpected type.

2. Objectives

- To classify the different categories of failure that can occur during an API call.
- To understand HTTP status codes and map them to appropriate program behaviour.
- To implement structured error handling using try–catch blocks, status checks and timeouts.
- To access deeply nested JSON fields safely using optional chaining, default values and validation.
- To apply retry, fallback and logging strategies that improve the reliability of a client application.

3. Introduction

An Application Programming Interface (API) allows one program to request data or services from another program, usually across a network. Most modern web APIs follow the REST style and exchange data in JavaScript Object Notation (JSON) format. Because the request travels over a network and the response is produced by a system that the client does not control, an API call is inherently unreliable. The network may be unavailable, the server may be overloaded, the authentication token may have expired, or the response body may simply not contain the field that the client expects.

Error handling is the discipline of anticipating these failures and responding to them in a controlled manner instead of allowing the program to terminate abruptly. Safe JSON access is the closely related practice of reading values from a parsed response only after confirming that the intermediate objects actually exist. Together, these two practices decide whether an application degrades gracefully or fails visibly in front of the user.

4. Categories of API Failure

Category	Typical Cause	Recommended Response
Network error	No internet, DNS failure, connection refused, request timeout	Retry with backoff; show an offline message
Client error (4xx)	Bad request body, invalid token, missing permission, wrong ID	Do not retry; correct the request or prompt the user
Server error (5xx)	Unhandled exception, service outage, gateway failure	Retry a limited number of times; then show a fallback
Parsing error	Response is HTML or plain text instead of valid JSON	Catch the parse exception; log the raw body
Schema error	Valid JSON, but an expected key is absent or of the wrong type	Use defaults and validation; never assume structure

5. Common HTTP Status Codes

Code	Meaning	Action by the Client
200	OK – Request succeeded	Parse and use the body
201	Created – Resource created	Read the new resource id
204	No Content – Success, empty body	Do not attempt to parse JSON
400	Bad Request – Malformed input	Fix and resend; show validation errors
401	Unauthorized – Missing or invalid credentials	Refresh the token or log in again
403	Forbidden – Authenticated but not permitted	Show an access-denied message
404	Not Found – Resource does not exist	Show an empty state; do not retry
429	Too Many Requests – Rate limit exceeded	Wait for Retry-After, then retry
500 / 502 / 503	Server or gateway failure	Retry with exponential backoff

6. Error Handling in JavaScript (Fetch API)

The fetch function rejects its promise only for network-level failures. A response carrying status 404 or 500 is still considered a fulfilled promise, so the status must be checked explicitly through the response.ok property. The example below combines a timeout, a status check, a guarded parse step and a typed error object.

```
async function getUser(id, timeoutMs = 8000) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), timeoutMs);
  try {
    const res = await fetch(`https://api.example.com/users/${id}`, {
      signal: controller.signal,
      headers: { 'Accept': 'application/json' }
    });
    if (!res.ok) {
      const detail = await res.text();
      throw new ApiError(res.status, detail);
    }
    const type = res.headers.get('content-type') || '';
    if (!type.includes('application/json')) {
      throw new ApiError(res.status, 'Expected JSON, received ' + type);
    }
    return await res.json();
  } catch (err) {
    if (err.name === 'AbortError') throw new ApiError(0, 'Request timed out');
    if (err instanceof ApiError) throw err;
    throw new ApiError(0, 'Network failure: ' + err.message);
  } finally {
    clearTimeout(timer);
  }
}

class ApiError extends Error {
  constructor(status, message) {
    super(message);
    this.name = 'ApiError';
    this.status = status;
  }
}
```

7. Safe JSON Access

Once a response has been parsed, the resulting object is still untrusted. Writing data.user.address.city will raise a TypeError the moment address is absent or null. Safe access replaces this assumption with an explicit check at every level.

7.1 Optional Chaining and Nullish Coalescing

The optional chaining operator ?. short-circuits to undefined as soon as any link in the chain is null or undefined. The nullish coalescing operator ?? then supplies a default. Unlike ||, it treats 0, empty string and false as valid values.

```
// Unsafe - throws if any level is missing
const city = data.user.address.city;

// Safe - resolves to a default instead of throwing
const city = data?.user?.address?.city ?? 'Not available';
const orders = data?.user?.orders ?? [];
const count = data?.stats?.count ?? 0;
const label = data?.stats?.count || 'none'; // 0 becomes 'none'
const first = data?.items?.[0]?.name ?? 'Untitled';
data.onLoad?.();
```

7.2 A Generic Safe-Get Helper

When the same defensive pattern is required in many places, a small helper function keeps the calling code readable and centralises the logic.

```
function safeGet(obj, path, fallback = null) {
  if (obj === null || typeof obj !== 'object') return fallback;
  const keys = Array.isArray(path) ? path : path.split('.');
  let current = obj;
  for (const key of keys) {
    if (current === null || current === undefined) return fallback;
    current = current[key];
  }
  return current === undefined ? fallback : current;
}

safeGet(data, 'user.address.city', 'Unknown');
safeGet(data, ['items', 0, 'price'], 0);
```

7.3 Validating Types Before Use

Presence alone is not sufficient. A field may exist but hold a string where a number was expected, or an object where an array was expected. Validating the type before use prevents a second class of runtime failure.

```
function normaliseUser(raw) {
  const u = (raw && typeof raw === 'object') ? raw : {};
  return {
    id: typeof u.id === 'number' ? u.id : -1,
    name: typeof u.name === 'string' && u.name.trim() ? u.name : 'Anonymous',
    email: typeof u.email === 'string' ? u.email.toLowerCase() : '',
    tags: Array.isArray(u.tags) ? u.tags : [],
    active: u.active === true
  };
}

const user = normaliseUser(await getUser(42));
```

8. Equivalent Handling in Python

The same principles apply in Python. The requests library raises `RequestException` for transport failures, while `.json()` raises a decoding error for an invalid body. Nested access is made safe with chained `.get()` calls that always return a dictionary.

```
import requests
from requests.exceptions import RequestException, Timeout, HTTPError

def get_user(user_id):
    try:
        res = requests.get(f'https://api.example.com/users/{user_id}', timeout=8)
        res.raise_for_status()
        return res.json()
    except Timeout:
        print('The request timed out.')
    except HTTPError as e:
        print(f'Server returned {e.response.status_code}')
    except ValueError:
        print('Response was not valid JSON.')
    except RequestException as e:
        print(f'Network error: {e}')
    return None

data = get_user(42) or {}
city = data.get('user', {}).get('address', {}).get('city', 'Unknown')
orders = data.get('user', {}).get('orders') or []
```

9. Retry with Exponential Backoff

Transient failures such as 429 and 503 usually resolve on their own. Retrying immediately, however, increases the load on an already struggling server. Exponential backoff doubles the waiting period after each attempt, and a small random jitter prevents many clients from retrying at the same instant.

```

async function requestWithRetry(fn, attempts = 3, base = 500) {
  for (let i = 0; i < attempts; i++) {
    try {
      return await fn();
    } catch (err) {
      const retryable = err.status === 429 || err.status >= 500 || err.status === 0;
      if (!retryable || i === attempts - 1) throw err;
      const wait = base * 2 ** i + Math.random() * 200;
      await new Promise(r => setTimeout(r, wait));
    }
  }
}

```

10. Best Practices

- Check the status code before parsing; a failed response often carries an HTML error page rather than JSON.
- Always set a timeout so that a hanging request cannot freeze the application indefinitely.
- Catch parsing errors separately from transport errors, because they require different corrective action.
- Never trust the shape of a response – use optional chaining, defaults and type checks at every level of nesting.
- Prefer ?? over || when zero, false or an empty string are legitimate values.
- Retry only idempotent operations, and only for transient status codes such as 429, 502 and 503.
- Show the user a clear, non-technical message while logging the full technical detail for developers.
- Never expose stack traces, tokens or internal URLs in a message shown to the end user.
- Fail fast at the boundary: validate and normalise the response once, so that the rest of the code works with trusted data.
- Use a schema validator such as Zod, Joi or Pydantic when the response structure is large or business critical.

11. Result

The different categories of API failure were studied, the corresponding HTTP status codes were mapped to appropriate client behaviour, and defensive techniques for reading values from a JSON response were implemented successfully in both JavaScript and Python.

12. Conclusion

Error handling and safe JSON access are not optional refinements added at the end of development; they are a core part of writing a dependable client. A program that checks the status code, guards its parsing step, validates the shape of the data it receives and retries only where retrying is meaningful will continue to behave predictably even when the service it depends on does not. The result is software that fails safely, explains itself clearly, and preserves the trust of the person using it.